全国高校 AI Web 开发挑战赛 · B 组 · 省赛作品

芯诊 AI

证据驱动的 STM32 USART 故障诊断工作台

技术文档（省赛提交材料）

版本：2026-10-25

形态：Web 应用 · 规则层 + 确定性工具 + RAG + 本地大模型解释

可追溯 · 不硬猜 · 可重复 · 证据纪律

芯诊 AI · 技术文档

> 版本：2026-10-25（Phase 0–7 全部软件侧交付后整理）
> 适用读者：评委、开发者、使用者
> 本文档所有结论遵守项目证据纪律：系统内无任何 `[实测-硬件]` 结论（硬件实测 0/6，待用户上板回填）。
> 固定评测集仅 9 用例，所有评测数字均为**趋势口径**，不宣称任何准确率。

---

1. 产品概述

芯诊 AI 是一个证据驱动的 STM32 USART 故障诊断 Web 工作台： 用户粘贴代码 / 编译日志 / 串口抓帧，系统用确定性工具提取证据、检索自建知识库、 按规则推理出**带证据链的候选根因**，并给出**可在真机上执行的验证步骤**； 用户回填实测结果后，系统重排结论并出具**七章结构化诊断报告。**

核心价值不是"更准"，而是 **可追溯、不硬猜、可重复**：

- 每条结论都能点回它的证据（源码行、寄存器配置、波特率计算、抓帧统计）；
- 材料不足时明确说"不足"，列出待补材料，而不是给一个无法验证的猜测；
- 同一个任务重跑，同一份证据得到同一份结论（确定性工具，无随机）。

1.1 核心能力

能力	说明
证据驱动诊断	规则层（23 条 UART 结构化规则）+ 8 工具 + BM25 检索自建 40 张知识卡片，端到端留痕
多步验证工作流	候选根因 → 验证步骤 → 用户在真机上回填结果 → 系统据此重排并出报告
可追溯报告	每个结论都挂证据与工具调用记录；重新生成保留历史版本
模型安全可控	三级降级链（云端配置位→本地 Ollama→规则层）、输出守卫、模型无诊断权
零部署依赖	SQLite 单文件 + 本地服务，clone 即跑；密钥零入库（`.env.example`）

2. 需求分析

2.1 目标用户

嵌入式初学者、高校实训学生、维护一线工程师——遇到"串口不通/乱码/无输出"类故障时， 需要一个**能解释为什么**的诊断工具，而不是一个可能自信地给出错误结论的聊天框。

2.2 功能需求（九步主流程）

1. 新建诊断任务，粘贴源码（+ 可选编译日志、实测波特率、串口抓帧）；
2. 描述故障现象；

3. 系统判定材料是否充分（无 USART 证据 → 明确"材料不足"并列出待补清单）；
4. 8 个确定性工具依次执行并留痕；
5. 规则匹配 + 症状加权 → 候选根因排序（含参考卡片）；
6. 每个候选生成可执行的验证步骤（预期现象）；
7. SSE 实时推送全过程；
8. 用户回填实测结果 → 系统重排候选；
9. 生成七章结构化诊断报告（可重新生成、保留版本）。

2.3 非功能需求

- **可追溯**：所有结论挂证据与工具调用记录；
- **可控**：模型输出恒 `[推断]`、有输出守卫、失败降级不中断；
- **安全**：不执行用户代码、密钥零入库、无 SQL 注入/XSS 面；
- **零部署成本**：SQLite + 本地服务，提交即可运行；
- **可用性**：空 / 加载 / 错误 / 恢复四态齐全，演示断网可降级。

2.4 MVP 边界（明确排除）

- 仅 STM32F103C8T6 + USART + 标准库；
- 不做多芯片/多协议扩展；不做多模态输入；不做云端推理依赖；
- 排除项与理由详见 `docs/Phase1_产品需求文档.md`。

3. 系统架构

3.1 分层



3.2 数据流（一次诊断）

1. 用户提交 `source_code` + `symptom_text`（+ 可选 `build_log`、波特率实测值）；
2. 引擎做**材料完整性前置判定**：无 USART 证据 → 直接 `unknown`，列待补材料；
3. 有证据 → 8 工具依次提取事实（时钟使能 / GPIO 模式 / 波特率误差 / ISR 体 / 抓帧统计…）；
4. L1 规则匹配 + 症状加权 → 候选根因排序；BM25 检索挂参考卡片；
5. 每个候选生成**验证步骤**（哪些能在真机做、预期现象是什么）；
6. SSE 实时推送：任务状态 / 工具调用 / 候选更新 / 证据落库；
7. 用户在真机执行验证并回填 `observed` / `match`；
8. 引擎按回填重排候选，生成七章报告（含硬件实测记录章，全部带证据标签）。

3.3 关键设计决策（ADR 摘要）

决策	理由
规则层为先、模型只解释	确定性工具不产生幻觉；模型负责语言表达，二者权责分离

自研状态机而非 LangChain	编排逻辑简单、可测试、零依赖
BM25 中文 2-gram + ASCII 整词	自建语料仅 40 条，词级切分命中差；评测中参考命中 45 次
SQLite + 本地文件	零部署、提交即跑
三级降级链	无网 / 无模型时功能不塌，演示容错
证据标签入类型系统 + DB CHECK	在编译期和存储层双重杜绝"把推断写成实测"

4. 证据纪律与安全

4.1 六档证据标签

标签	含义	谁能产生
`[实测-硬件]`	用户上板回填的原始记录	**仅用户**（T-07 抓帧回填通道）
`[实测-仿真]`	Keil 仿真 / 模拟环境运行结果	预留
`[静态分析]`	代码 / 配置 / 日志静态检查	工具层（T-01…T-06）
`[推断]`	模型基于证据的推理	**仅模型**（恒带此标签）
`[假设]`	无证据支撑的猜测	系统不产出（语料中知识等级标注）
`[待验证]`	计划中未执行	阈值 / 现象对照表

落地机制：Python 类型系统构造校验 → 接口层证据标签白名单 → SQLite `CHECK` 约束 → 报告渲染器将标签渲染为徽章。`_tag()` 对非法标签直接抛异常，**不静默降级。**

4.2 模型安全

- **模型无诊断权**：不参与根因判定、不改评分、不新增候选；
- **输出守卫**：结构校验（JSON / 字段类型）、长度截断（摘要 400 字符、步骤 4 条 × 120 字符）、校验失败 → 降级规则层并记录原因；
- **恒 `[推断]`**：模型输出一律标记推断，UI 与报告端到端强制；
- **本地优先**：默认仅连 `127.0.0.1:11434`；云端仅配置位（`.env.example` 中留空即不启用），密钥只走环境变量。

4.3 应用安全（security_audit 13 项全过）

- 不执行用户代码：诊断工具纯静态解析；
- 无 SQL 注入面：SQLite 访问全部参数化；
- 密钥零入库：`.env` 排除、扫描器验证无泄漏；
- XSS：React 默认转义 + 报告为受控 Markdown 子集渲染；
- 上传限制：单文件解析上限 512 KB。

5. 诊断工具清单（8 个）

编号	工具	输入 → 事实	证据等级
T-01	parse_build_log	编译/链接日志 → 错误行、缺定义	[静态分析]
T-02	analyze_c_code	C 源码 → 外设/引脚/时钟调用事实	[静态分析]
T-03	check_uart_config	源码 → UART/GPIO/RCC/NVIC 一致性	[静态分析]
T-04	calc_baud_error	配置 → 波特率误差%（阈值 2%，[待验证]）	[静态分析]
T-04r	infer_actual_fck	实测波特率 → 反推实际总线时钟	[推断]
T-05	search_reference	症状/证据 → BM25 相关卡片	[静态分析]
T-06	analyze_isr_body	ISR 体 → 清标志/进/出中断检查	[静态分析]
T-07	parse_serial_capture	抓帧 → EMPTY/GARBLED/NORMAL 统计	**唯一可产** [实测-硬件]
T-08	llm_assist（按需）	证据摘要 → 大白话解释 + 补充验证建议	恒 [推断]

6. 语料与检索（全部自撰，无版权风险）

- **L1 规则库**：`docs/corpus/l1_rules_uart.json`，23 条，覆盖 6 类故障场景 + 通用规则；
- **L2 知识卡片**：`docs/corpus/cards/`，40 条 × 6 主题（时钟/波特率/GPIO/重映射/NVIC/printf）；
- **现象对照表**：`phenomena_map.json`，6 条，全部标 [推断]，待硬件实测回填校准；
- **检索**：BM25（中文 2-gram 切分 + ASCII 整词），语料小、可追溯；
- **知识来源**：公有技术事实（芯片手册章节级引用，非原文转载）+ 自撰工程经验；
- **消融结论**：去掉 RAG → 命中率不变但参考 45→0（RAG 贡献=**可追溯性**，非准确率）。

7. AI 选型理由与集成方式

7.1 为什么本地小模型 + 规则层，而不是只做大模型

维度	纯 LLM 方案	芯诊 AI（规则层+工具+RAG+LLM 解释）
材料不足	可能硬猜根因（评测 3 负例中踩坑）	明确"材料不足"并列出待补清单（0 误报）
超范围材料	可能把 I2C 硬套成 UART 结论	识别范围外，拒绝套结论
可追溯	无证据链	每条结论可点回证据
稳定性	输出抖动（同输入多次结果不同）	确定性工具，同证据同结论
承载实测	无法产生 [实测-硬件]	T-07 抓帧回填通道，实测由用户产生
成本/隐私	云端 API 依赖网络与密钥	本地运行、零密钥、断网可演示

7.2 本地模型选型（三模型基线横向对比，2026-10-25）

固定评测集 9 用例（6 正 + 3 负），同一批材料、同一候选空间、每模型 3 次采样（趋势口径）：

指标	规则层+工具	qwen2.5:7b	qwen2.5-coder:7b	qwen2.5:3b
正例 Top-1 全中	6/6	6/6	6/6	5/6
负例误报（任一采样）	**0**	2	**0**	1
输出稳定（×3 一致）	确定性	7/9	**9/9**	8/9
平均单次耗时	<10 ms	2736 ms	2247 ms	895 ms

选型结论：默认模型为 `qwen2.5-coder:7b`（正例不输、负例防坑最好、输出最稳、 还答对了唯一的编译错误用例）； `qwen2.5:3b` 保留为弱机/快速备选； `qwen2.5:7b` 作备选。 完整逐条明细见 `docs/Phase6_模型选型对比.md`。

> ⚠️ 口径：样本仅 9 例，只作趋势；**不宣称"比大模型更准"**。
> 真实差异是可追溯、不硬猜、超范围不套结论、能承载 `[实测-硬件]`。

7.3 集成方式与优化策略

- **集成**：标准库 `urllib` 直连 Ollama HTTP API（**零第三方依赖**）；三级降级链 云端配置位 → 本地 → 规则层，健康检查带缓存（20s），失败自动降级并留痕；
- **提示词**：分层设计——系统提示（角色 + 证据纪律）、任务提示（故障类型）、 上下文注入（代码片段 + 日志 + 检索结果）；
- **优化**：输出守卫截断与校验；温度 0.2 抑制发散；冷启动靠健康缓存与演示预热规避；
- **评估**：27 项模型层测试（离线替身 + 在线真实调用）+ 16 项模型链路冒烟。

8. 测试与评测

8.1 自动化测试（8 套件一键回归）

命令：`cd apps/server && python run_all_tests.py`（子脚本退出码非 0 即 FAIL；依赖不可达判 SKIP 不判通过）。

套件	数量	结果
工具层单测（8 工具确定性行为）	52	✅ 52/52
故障样例自检（12 个自撰样例）	24	✅ 24/24
模型层测试（守卫/降级/在线真实调用）	27	✅ 27/27
固定评测集（6 正 + 3 负）	9	✅ 判定通过
边界与异常（超大/空输入/非法标签/注入）	16	✅ 16/16
仓库合规审计（密钥/版权/XSS）	13	✅ 13/13（1 条正当 WARN）

端到端 HTTP 冒烟	62	✅ 62/62
模型解读链路冒烟	16	✅ 16/16

8.2 固定评测集与指标

- 6 正例：C1 时钟未使能 / C2 波特率不符 / C3 TX 引脚模式 / C4 重映射无 AFIO / C5 NVIC 未挂接 / C6 printf 未重定向；
- 3 负例：N-01 材料严重不足 / N-02 编译错误被描述成硬件故障 / N-03 I2C 超范围硬套；
- 指标：Top-1 命中、负例误报、证据可追溯、自动幻觉筛查。

指标	结果
正例 Top-1 命中	6/6
负例误报	0/3
证据可追溯	8/8
自动幻觉筛查	0 条可疑

评测抓出并修复的真缺陷（诚实记录）：① 空材料硬给根因 → 新增材料前置判定；② I2C 被硬套 UART 结论 → 编排层超范围 followup；③ 材料不足缺引导 → 待补清单；④ 幻觉筛查器自身 2 处误报 → 修正匹配逻辑。

8.3 消融

实验	结果	结论
去掉 RAG	命中率不变，参考 45→0	RAG 贡献=可追溯，非准确率
去掉症状加权	0/9 排序改变	当前测试集影响为 0，不宣称"症状驱动排序"
关键词朴素基线	3 负例误报 2	复现 N-01/N-03 两个坑

9. 部署与运行

9.1 环境要求

- Python 3.11+（开发用 3.13）；Node.js 18+（开发用 22.x）；
- （可选）本地 Ollama `http://127.0.0.1:11434`，已拉取 `qwen2.5-coder:7b`（默认）等；
- 没有模型也不影响：自动降级纯规则层。

9.2 启动

```
bash
# 后端
cd apps/server
python -m venv .venv && .venv/Scripts/activate
pip install fastapi "uvicorn[standard]" pydantic
python -m uvicorn app.main:app --host 127.0.0.1 --port 8787

# 前端（另开终端）
cd apps/web
npm install
npm run dev          # http://127.0.0.1:5173

# 一键回归
cd apps/server && python run_all_tests.py
```

9.3 配置

密钥与阈值全部走环境变量：`apps/server/.env.example`（`XZ_*` 前缀，与 `config.py` 一致）。 云端密钥留空即不启用；`.env` 已被 `.gitignore` 排除，绝不入库。

10. 技术创新点与应用创新点

10.1 技术创新点

1. **证据纪律系统化**：六档证据标签从 Python 类型系统、接口白名单到 SQLite CHECK 约束三层落地； 模型恒 `[推断]`、无诊断权——AI 输出安全与可控性内建在架构里，而非提示词约束；
2. **规则层+工具+模型权责分离**：确定性工具查证、规则层判定、模型只解释， 模型幻觉被结构性隔离；
3. **多步验证闭环**：候选 → 验证步骤 → 真机回填 → 重排 → 报告， 诊断是可复核的实验过程而非一次问答；
4. **自研轻量 LLM 接入**：标准库直连 Ollama、零第三方依赖、三级降级链， 断网也能完整演示；
5. **自建领域语料 + 轻量 RAG**：23 规则 + 40 卡片全部自撰（零版权风险）， BM25 中文 2-gram 检索，贡献可追溯；
6. **材料不足识别**：把"没看到"与"有错"区分开，评测负例 0 误报。

10.2 应用创新点

1. 以「诊断任务」为中心的证据面板工作台（非聊天框）：候选卡片、支持/反对证据、 可点击验证步骤、实测回填后自动更新判断；
2. 硬件实测证据由用户上板产生、系统承载并重排——AI 不越权"宣布实测"；
3. 降低嵌入式调试门槛：高校实训与一线排障的"师傅级"证据链可复制。

11. 总结与展望

总结：本项目用"确定性工具查证 + 规则推理 + 本地模型解释"的组合， 把 STM32 USART 故障诊断从"猜答案"变成"查证据"，形成可追溯、不硬猜、可重复的闭环； 软件侧 8 套件全绿、9 用例评测达标、安全合规通过、默认模型经三模型基线选定。

展望：

- **硬件实测回填**（提交前重点）：C1–C6 上板抓帧回填，产生系统首批 `[实测-硬件]` 证据并校准阈值；
- **协议扩展**：USART2/3、SPI、I2C、CAN 的规则库与工具逐步扩展（MVP 边界内保持不硬套）；
- **多模态**：串口抓帧波形图、示波器截图识别（当前 P1-D 决策未投入，作为后续方向）；
- **云端可选**：降级链第①级接入云端大模型用于更复杂场景（配置位已留）；
- **在线演示**：部署公网版本供评审访问（可选加分项）。

12. 目录速查

```
apps/server/app/engine/     8 工具 + orchestrator
apps/server/app/corpus/     规则/卡片加载 + BM25
apps/server/app/llm/        客户端 / 提示词 / 守卫 / 降级
apps/server/app/            main.py（路由）/ db.py（13 表）
apps/server/eval/           评测集 / 消融 / 边界 / 安全 / LLM 基线 / run_all_tests.py
apps/server/samples/        12 个自撰样例
apps/web/src/               前端（components/ 三栏 + 报告渲染器）
docs/                       方案 / 验收 / 语料 / 风险 / 决策记录
deliverables/               对外交付材料
firmware/baseline_usart1/   基准工程（自撰）
```